



Pichulman Miguel Angel

BASE DE DATOS 2
TRABAJO PRACTICO 2
Modelado y Diseño de Bases NoSQL (MongoDB)

1) Una plataforma educativa almacena Cursos, Estudiantes, y Evaluaciones. Cada curso tiene muchos estudiantes, y cada estudiante puede realizar varias evaluaciones dentro de un curso.

a) Diseñar el documento de Curso aplicando un patrón híbrido, donde se embeban los datos esenciales de las evaluaciones (promedio y cantidad), pero las evaluaciones completas se almacenen en otra colección llamada evaluaciones.

-Estructura JSON del documento Curso

```
{
  "_id": "C1",
  "nombre": "Bases de Datos 2",
  "descripcion": "Bases de datos NoSQL",
  "estudiantes": [
    {
      "estudiante_id": "E1",
      "nombre": "Ana Pérez",
      "email": "ana.perez@example.com",
      "evaluaciones_resumen": {
        "cantidad": 3,
        "promedio": 8.7
      }
    },
    {
      "estudiante_id": "E2",
      "nombre": "Carlos Gómez",
      "email": "carlos.gomez@example.com",
      "evaluaciones_resumen": {
        "cantidad": 2,
        "promedio": 7.5
      }
    }
  ]
}
```

-Estructura JSON de la colección evaluaciones

```
{
  "_id": "E1",
  "curso_id": "C1",
  "estudiante_id": "E1",
  "fecha": "2026-04-10",
  "tipo": "Examen Parcial",
  "nota": 9
}
```



Pichulman Miguel Angel

b) Explicar brevemente por qué el patrón híbrido es más adecuado que usar solo embebido o solo referencia en este caso.

Consultas rápidas: el curso ya tiene embebido el promedio y cantidad de evaluaciones por estudiante.

Flexibilidad: las evaluaciones completas se guardan en otra colección, evitando documentos demasiado grandes.

Escalabilidad: si un estudiante tiene cientos de evaluaciones, no saturamos el documento del curso.

2) Conectarse a mongosh y crear la base de datos 'plataforma_educativa'. Implementar el diseño propuesto.

use plataforma_educativa

a) Crear la colección cursos e insertar 2 cursos con datos embebidos de resumen de evaluaciones.

```
db.cursos.insertMany([
  {
    _id: "C1",
    nombre: "Bases de Datos 2",
    descripcion: "Bases de datos NoSQL",
    estudiantes: [
      {
        estudiante_id: "E1",
        nombre: "Ana Perez",
        email: "ana.perez@example.com",
        evaluaciones_resumen: { cantidad: 3, promedio: 8.7 }
      },
      {
        estudiante_id: "E2",
        nombre: "Carlos Gomez",
        email: "carlos.gomez@example.com",
        evaluaciones_resumen: { cantidad: 2, promedio: 7.5 }
      }
    ]
  },
  {
    _id: "C2",
    nombre: "Programacion 3",
    descripcion: "HTML, CSS y JS",
    estudiantes: [
      {
        estudiante_id: "E3",
        nombre: "Lucia Fernandez",
        email: "lucia.fernandez@example.com",
        evaluaciones_resumen: { cantidad: 4, promedio: 9.2 }
      }
    ]
  }
])
```



Pichulman Miguel Angel

```
plataforma_educativa> db.cursos.insertMany([
  {
    _id: "C1",
    nombre: "Bases de Datos 2",
    descripcion: "Bases de datos NoSQL",
  },
  {
    _id: "C2",
    nombre: "Programacion 3",
    descripcion: "HTML, CSS y JS",
  }
])
{ acknowledged: true, insertedIds: { '0': 'C1', '1': 'C2' } }
plataforma_educativa>
```

b) Crear la colección 'evaluaciones' con las evaluaciones detalladas que referencian a los cursos.

```
db.evaluaciones.insertMany([
  {
    curso_id: "C1",
    estudiante_id: "E1",
    fecha: "2026-04-10",
    tipo: "Examen Parcial",
    nota: 9.0
  },
  {
    curso_id: "C1",
    estudiante_id: "E1",
    fecha: "2026-04-20",
    tipo: "Trabajo Práctico",
    nota: 8.5
  },
])
```



Pichulman Miguel Angel

```
{  
  curso_id: "C2",  
  estudiante_id: "E2",  
  fecha: "2026-04-15",  
  tipo: "Examen Final",  
  nota: 9.5  
}  
])
```

```
plataforma_educativa> db.evaluaciones.insertMany([  
  {  
    curso_id: "C1",  
    estudiante_id: "E1",  
    fecha: "2026-04-10",  
    tipo: "Examen Parcial",  
    nota: 9.0  
  },  
  {  
    curso_id: "C1",  
    estudiante_id: "E1",  
    fecha: "2026-04-20",  
    tipo: "Trabajo Práctico",  
    nota: 8.5  
  },  
  {  
    curso_id: "C2",  
    estudiante_id: "E2",  
    fecha: "2026-04-15",  
    tipo: "Examen Final",  
    nota: 9.5  
  }  
])  
{  
  acknowledged: true,  
  insertedIds: {  
    '0': ObjectId('69f248d5b636efed653682d1'),  
    '1': ObjectId('69f248d5b636efed653682d2'),  
    '2': ObjectId('69f248d5b636efed653682d3')  
  }  
}  
plataforma_educativa> |
```

c) Realizar una consulta que obtenga un curso específico con su resumen de evaluaciones.

```
db.cursos.find(  
  { nombre: "Bases de Datos 2" },  
  { _id: 0, nombre: 1, estudiantes: 1 }  
)<pre>
.pretty()
```



Pichulman Miguel Angel

```
plataforma_educativa> db.cursos.find(
|   { nombre: "Bases de Datos 2" },
|   { _id: 0, nombre: 1, estudiantes: 1 }
| ).pretty()
[
  {
    nombre: 'Bases de Datos 2',
    estudiantes: [
      {
        estudiante_id: 'E1',
        nombre: 'Ana Perez',
        email: 'ana.perez@example.com',
        evaluaciones_resumen: { cantidad: 3, promedio: 8.7 }
      },
      {
        estudiante_id: 'E2',
        nombre: 'Carlos Gomez',
        email: 'carlos.gomez@example.com',
        evaluaciones_resumen: { cantidad: 2, promedio: 7.5 }
      }
    ]
  }
]
```



Pichulman Miguel Angel

d) Realizar una consulta que obtenga todas las evaluaciones detalladas de un curso específico.

```
db.evaluaciones.find(  
  { curso_id: "C1" }  
)
```

```
plataforma_educativa> db.evaluaciones.find(  
|   { curso_id: "C1" }  
| ).pretty()  
[  
  {  
    _id: ObjectId('69f248d5b636efed653682d1'),  
    curso_id: 'C1',  
    estudiante_id: 'E1',  
    fecha: '2026-04-10',  
    tipo: 'Examen Parcial',  
    nota: 9  
  },  
  {  
    _id: ObjectId('69f248d5b636efed653682d2'),  
    curso_id: 'C1',  
    estudiante_id: 'E1',  
    fecha: '2026-04-20',  
    tipo: 'Trabajo Práctico',  
    nota: 8.5  
  }  
]  
plataforma_educativa> |
```



Pichulman Miguel Angel

e) Realizar una agregación que calcule el promedio de notas por curso.

```
db.evaluaciones.aggregate([
  {
    $group: {
      _id: "$curso_id",
      promedioNotas: { $avg: "$nota" },
      cantidadEvaluaciones: { $sum: 1 }
    }
  }
])
```

```
plataforma_educativa> db.evaluaciones.aggregate([
|   {
|     $group: {
|       _id: "$curso_id",
|       promedioNotas: { $avg: "$nota" },
|       cantidadEvaluaciones: { $sum: 1 }
|     }
|   }
| ])
[
|   { _id: 'C2', promedioNotas: 9.5, cantidadEvaluaciones: 1 },
|   { _id: 'C1', promedioNotas: 8.75, cantidadEvaluaciones: 2 }
| ]
plataforma_educativa> |
```



Pichulman Miguel Angel

ACTIVIDAD 2 : Relación estudiante-tutor - Muchos a Muchos

- 1) Cada estudiante puede tener uno o más tutores, y un tutor puede acompañar a varios estudiantes. Para cada tutoría se guarda la fecha de inicio y el tipo de acompañamiento (académico, motivacional o técnico).

- a) Identificar qué tipo de relación se forma entre Estudiante y Tutor (uno-a-uno, uno-a-muchos, muchos-a-muchos).

La relación es Mucho a Muchos (N:M)

- b) Indicar qué patrón conviene usar: embebido, referencia o colección asociativa. Justificar tu respuesta.

Conviene utilizar el patrón Colección Asociativa dado que:

Atributos en la relación: Necesitamos guardar datos específicos del vínculo (fecha_inicio y tipo_acompañamiento), los cuales no pertenecen puramente ni al estudiante ni al tutor.

Escalabilidad: Evita que los documentos crezcan indefinidamente.

Flexibilidad de consulta: Permite consultar fácilmente tanto "qué tutores tiene un alumno" como "qué alumnos tiene un tutor" sin duplicar información compleja.

- c) Diseñar la estructura de documentos que represente esta relación, incluyendo los campos 'fecha_inicio' y 'tipo_acompañamiento'.

Colección estudiantes: _id, nombre, carrera.

Colección tutores: _id, nombre, area_expertise.

Colección tutorías: _id, estudiante_id (FK), tutor_id (FK), fecha_inicio, tipo_acompañamiento.

2) Implementar el diseño propuesto usando el patrón de colección asociativa.

- a) Crear la colección 'estudiantes' con al menos 3 estudiantes.

```
db.estudiantes.insertMany([
  { _id: 1, nombre: "Ana García", carrera: "Sistemas" },
  { _id: 2, nombre: "Lucas Martínez", carrera: "Diseño" },
  { _id: 3, nombre: "Carla Rodríguez", carrera: "Sistemas" }
]);
```

- b) Crear la colección 'tutores' con al menos 2 tutores.

```
db.tutores.insertMany([
  { _id: 101, nombre: "Ing. Carlos Pérez", area_expertise: "Backend" },
  { _id: 102, nombre: "Lic. Marta Gómez", area_expertise: "Metodologías" }
]);
```

- d) Crear la colección asociativa 'tutorías' que relacione estudiantes con tutores.

```
db.tutorias.insertMany([
  {
    estudiante_id: 1,
    tutor_id: 101,
    fecha_inicio: new Date("2026-03-01"),
    tipo_acompañamiento: "técnico"
  },
  {
```


**TECNICATURA UNIVERSITARIA
EN PROGRAMACIÓN
A DISTANCIA**



Pichulman Miguel Angel

```
estudiante_id: 1,  
tutor_id: 102,  
fecha_inicio: new Date("2026-03-15"),  
tipo_acompanamiento: "académico"  
},  
{  
  estudiante_id: 2,  
  tutor_id: 101,  
  fecha_inicio: new Date("2026-04-01"),  
  tipo_acompanamiento: "técnico"  
},  
{  
  estudiante_id: 3,  
  tutor_id: 102,  
  fecha_inicio: new Date("2026-04-10"),  
  tipo_acompanamiento: "motivacional"  
}  
]);
```

e) Consultar todos los estudiantes de un tutor específico

```
db.tutorias.aggregate([  
  {  
    $match: { tutor_id: 101 }  
  },  
  {  
    $lookup: {  
      from: "estudiantes",  
      localField: "estudiante_id",  
      foreignField: "_id",  
      as: "detalle_estudiante"  
    }  
  },  
  {  
    $unwind: "$detalle_estudiante"  
  },  
  {  
    $project: {  
      _id: 0,  
      tutor_id: 1,  
      "nombre_estudiante": "$detalle_estudiante.nombre",  
      tipo_acompanamiento: 1,  
      fecha_inicio: 1  
    }  
  }  
]);
```



Pichulman Miguel Angel

```
< {  
  fecha_inicio: 2026-03-01T00:00:00.000Z,  
  tipo_acompanamiento: 'técnico',  
  nombre_estudiante: 'Ana García',  
  carrera: 'Sistemas'  
}  
{  
  fecha_inicio: 2026-04-01T00:00:00.000Z,  
  tipo_acompanamiento: 'técnico',  
  nombre_estudiante: 'Lucas Martínez',  
  carrera: 'Diseño'  
}
```

ACTIVIDAD 3 : Biblioteca digital - Recursos compartidos

1) Una biblioteca digital almacena Materias y Recursos (archivos PDF, videos o enlaces). Cada materia tiene muchos recursos, pero los recursos pueden ser reutilizados en distintas materias.

a) ¿Qué patrón de modelado usarías para vincular Materias y Recursos? (embebido, referencia o híbrido)

Se recomienda utilizar el patrón de referencia.

b) Explicar brevemente por qué el embebido NO sería adecuado en este caso.

Duplicación de datos: Si un mismo video de "Introducción a SQL" se usa en 10 materias, y el link del video cambia, tendrías que actualizarlo en 10 documentos distintos.

Inconsistencia: Corres el riesgo de que el mismo recurso tenga metadatos diferentes (un título distinto o una descripción vieja) en materias diferentes.

Tamaño del documento: Si una materia llegara a tener una cantidad masiva de recursos embebidos, el documento de la materia podría volverse pesado innecesariamente.



Pichulman Miguel Angel

c) Diseñar un ejemplo de documento Materia y uno de Recurso, indicando cómo se relacionan.

Documento Recurso:

```
{
  "_id": "501",
  "titulo": "Manual de NoSQL",
  "tipo": "pdf",
  "url": "https://drive.com/manual123"
}
```

Documento Materia:

```
{
  "_id": "Base de Datos",
  "nombre": "Bases de Datos 2",
  "recursos": [
    "501",
  ]
}
```

2) Implementación práctica.

a) Crear la colección 'recursos' con recursos que puedan ser compartidos.

```
db.recursos.insertMany([
  { _id: 501, titulo: "Manual de NoSQL", tipo: "PDF", url: "drive.com/manual123" },
  { _id: 502, titulo: "Intro a Modelado", tipo: "video", url: "youtube.com/v1" },
  { _id: 503, titulo: "Cheat Sheet JS", tipo: "enlace", url: "github.com/js" }
]);
```

b) Crear la colección 'materias' que referencie a los recursos.

```
db.materias.insertMany([
  { _id: "BD2", nombre: "Bases de Datos 2", recursos: [501, 502] },
  { _id: "PROG3", nombre: "Programación 3", recursos: [502, 503] },
  { _id: "SIST", nombre: "Sistemas Operativos", recursos: [501] }
]);
```

c) Consultar en cuántas materias se usa un recurso específico.

```
db.materias.countDocuments({ recursos: 501 });
```

```
> db.materias.countDocuments({ recursos: 501 });
< 2
```



Pichulman Miguel Angel

d) Listar todas las materias que usan un recurso específico.

```
db.materias.find({ recursos: 502 }, { nombre: 1, _id: 0 });
```

```
> db.materias.find({ recursos: 502 }, { nombre: 1, _id: 0 });
< {
  nombre: 'Bases de Datos 2'
}
{
  nombre: 'Programación 3'
}
Atlas atlas-cdbkjq-shard-0 [primary] trabajoPractico2>
```

e) Agregar un nuevo recurso a una materia existente

```
db.materias.updateOne(
  { _id: "BD2" },
  { $push: { recursos: 503 } }
);
```

```
> db.materias.updateOne(
  { _id: "BD2" },
  { $push: { recursos: 503 } }
);
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```